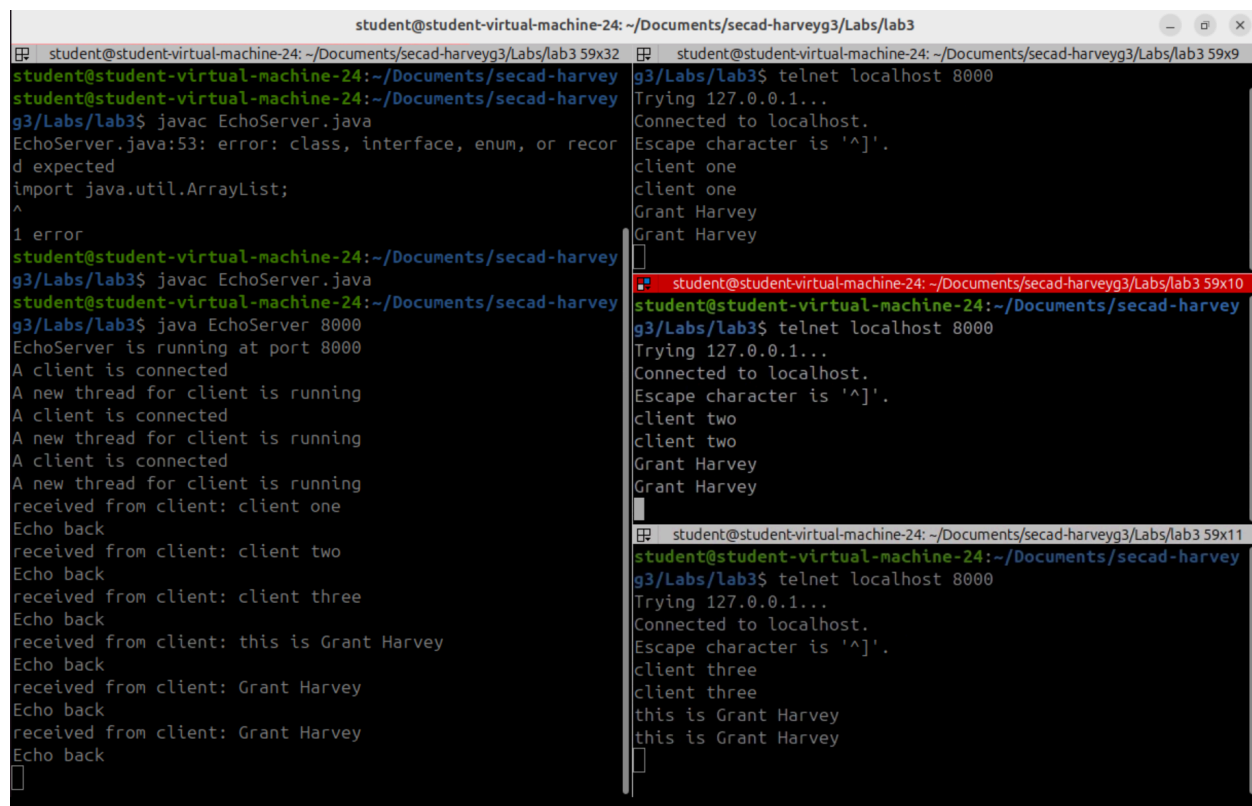


Grant Harvey
harveyg3@udayton.edu
CPS 475 - Software Development
Dr Phu Phung
Bitbucket: <https://bitbucket.org/granthsrvey/secad-harveyg3/src/main/>

Lab 03

Task 1 - Simple Multithreaded Programming in Java

a.



```
student@student-virtual-machine-24: ~/Documents/secad-harveyg3/Labs/lab3
student@student-virtual-machine-24: ~/Documents/secad-harveyg3/Labs/lab3 59x32
student@student-virtual-machine-24: ~/Documents/secad-harveyg3/Labs/lab3 59x9
g3/Labs/lab3$ javac EchoServer.java
EchoServer.java:53: error: class, interface, enum, or record expected
import java.util.ArrayList;
^
1 error
student@student-virtual-machine-24: ~/Documents/secad-harveyg3/Labs/lab3 59x10
g3/Labs/lab3$ javac EchoServer.java
student@student-virtual-machine-24: ~/Documents/secad-harveyg3/Labs/lab3 59x10
g3/Labs/lab3$ java EchoServer 8000
EchoServer is running at port 8000
A client is connected
A new thread for client is running
A client is connected
A new thread for client is running
A client is connected
A new thread for client is running
received from client: client one
Echo back
received from client: client two
Echo back
received from client: client three
Echo back
received from client: this is Grant Harvey
Echo back
received from client: Grant Harvey
Echo back
received from client: Grant Harvey
Echo back
received from client: Grant Harvey
g3/Labs/lab3$ telnet localhost 8000
Trying 127.0.0.1...
Connected to localhost.
Escape character is '^]'.
client one
client one
Grant Harvey
Grant Harvey
g3/Labs/lab3$ telnet localhost 8000
Trying 127.0.0.1...
Connected to localhost.
Escape character is '^]'.
client two
client two
Grant Harvey
Grant Harvey
g3/Labs/lab3$ telnet localhost 8000
Trying 127.0.0.1...
Connected to localhost.
Escape character is '^]'.
client three
client three
this is Grant Harvey
this is Grant Harvey
```

Source Code:

```
import java.net.*;
import java.io.*;
import java.util.ArrayList;

public class EchoServer {
    public static void main(String[] args) throws IOException {

        if (args.length != 1) {
            System.err.println("Usage: java EchoServer <port number>");
            System.exit(1);
        }

        int portNumber = Integer.parseInt(args[0]);
```

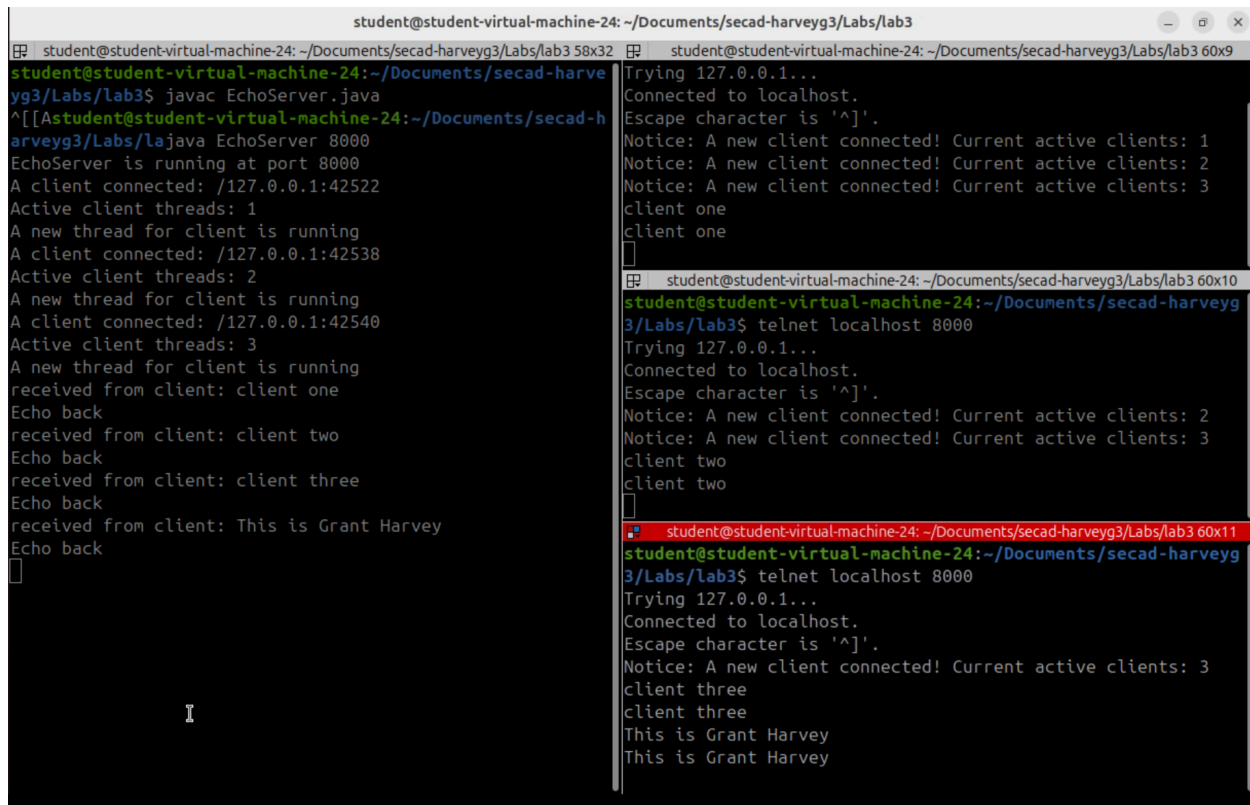
```

try {
    ServerSocket serverSocket = new ServerSocket(Integer.parseInt(args[0]));
    System.out.println("EchoServer is running at port " + Integer.parseInt(args[0]));
    while(true){
        Socket clientSocket = serverSocket.accept();
        System.out.println("A client is connected ");
        new EchoServerThread(clientSocket).start();
    }
} catch (IOException e) {
    System.out.println("Exception caught when trying to listen on port "
        + portNumber + " or listening for a connection");
    System.out.println(e.getMessage());
}
}
}

class EchoServerThread extends Thread {
    private Socket clientSocket = null;
    public EchoServerThread(Socket socket) {
        clientSocket = socket;
    }
    public void run(){
        System.out.println("A new thread for client is running");
        try{
            PrintWriter out = new PrintWriter(clientSocket.getOutputStream(), true);
            BufferedReader in = new BufferedReader(new InputStreamReader(clientSocket.getInputStream()));
            String inputLine;
            while ((inputLine = in.readLine()) != null) {
                System.out.println("received from client: " + inputLine);
                System.out.println("Echo back");
                out.println(inputLine);
            }
        }
        catch(IOException ioe) {
            System.out.println("Exception in threads:" + ioe.getMessage());
        }
    }
}
}

```

b.



The screenshot shows a terminal window with two panes. The left pane shows the compilation and execution of EchoServer.java. The right pane shows telnet connections to localhost on port 8000, demonstrating the server's response to client connections and messages.

```
student@student-virtual-machine-24: ~/Documents/secad-harveyg3/Labs/lab3
student@student-virtual-machine-24: ~/Documents/secad-harveyg3/Labs/lab3$ javac EchoServer.java
^[[Astudent@student-virtual-machine-24:~/Documents/secad-harveyg3/Labs/lab3$ java EchoServer 8000
EchoServer is running at port 8000
A client connected: /127.0.0.1:42522
Active client threads: 1
A new thread for client is running
A client connected: /127.0.0.1:42538
Active client threads: 2
A new thread for client is running
A client connected: /127.0.0.1:42540
Active client threads: 3
A new thread for client is running
received from client: client one
Echo back
received from client: client two
Echo back
received from client: client three
Echo back
received from client: This is Grant Harvey
Echo back
^[[

student@student-virtual-machine-24: ~/Documents/secad-harveyg3/Labs/lab3 58x32
student@student-virtual-machine-24: ~/Documents/secad-harveyg3/Labs/lab3 60x9
Trying 127.0.0.1...
Connected to localhost.
Escape character is '^'.
Notice: A new client connected! Current active clients: 1
Notice: A new client connected! Current active clients: 2
Notice: A new client connected! Current active clients: 3
client one
client one
^[[

student@student-virtual-machine-24: ~/Documents/secad-harveyg3/Labs/lab3 60x10
student@student-virtual-machine-24: ~/Documents/secad-harveyg3/Labs/lab3$ telnet localhost 8000
Trying 127.0.0.1...
Connected to localhost.
Escape character is '^'.
Notice: A new client connected! Current active clients: 2
Notice: A new client connected! Current active clients: 3
client two
client two
^[[

student@student-virtual-machine-24: ~/Documents/secad-harveyg3/Labs/lab3 60x11
student@student-virtual-machine-24: ~/Documents/secad-harveyg3/Labs/lab3$ telnet localhost 8000
Trying 127.0.0.1...
Connected to localhost.
Escape character is '^'.
Notice: A new client connected! Current active clients: 3
client three
client three
This is Grant Harvey
This is Grant Harvey
```

EchoServer.java Source Code:

```
import java.net.*;
import java.io.*;

public class EchoServer {
    public static void main(String[] args) throws IOException {

        if (args.length != 1) {
            System.err.println("Usage: java EchoServer <port number>");
            System.exit(1);
        }

        int portNumber = Integer.parseInt(args[0]);

        // Single shared instance for all client threads
        ThreadList sharedThreadList = new ThreadList();

        try {
            ServerSocket serverSocket = new ServerSocket(portNumber);
            System.out.println("EchoServer is running at port " + portNumber);

            while (true) {
                Socket clientSocket = serverSocket.accept();
                System.out.println("A client connected: " + clientSocket.getRemoteSocketAddress());

                // Pass the shared ThreadList instance to the new thread
                EchoServerThread clientThread = new EchoServerThread(clientSocket, sharedThreadList);

                // Add to list and start thread
                sharedThreadList.addThread(clientThread);
            }
        }
    }
}
```

```

        clientThread.start();
    }
} catch (IOException e) {
    System.out.println("Exception caught when trying to listen on port "
        + portNumber + " or listening for a connection");
    System.out.println(e.getMessage());
}
}
}

```

```

class EchoServerThread extends Thread {
    private Socket clientSocket;
    private ThreadList threadList;
    private PrintWriter out;

    public EchoServerThread(Socket socket, ThreadList threadList) {
        this.clientSocket = socket;
        this.threadList = threadList;
    }

    // Helper method used by ThreadList.sendToAll
    public void sendMessage(String message) {
        if (out != null) {
            out.println(message);
        }
    }

    public void run() {
        System.out.println("A new thread for client is running");
        try {
            out = new PrintWriter(clientSocket.getOutputStream(), true);
            BufferedReader in = new BufferedReader(new InputStreamReader(clientSocket.getInputStream()));

            // Optional: Broadcast join notification to all clients
            threadList.sendToAll("Notice: A new client connected! Current active clients: " + threadList.getNumberOfThreads());

            String inputLine;
            while ((inputLine = in.readLine()) != null) {
                System.out.println("received from client: " + inputLine);
                System.out.println("Echo back");
                out.println(inputLine);
            }
        } catch (IOException ioe) {
            System.out.println("Exception in thread: " + ioe.getMessage());
        } finally {
            // Clean up when client disconnects
            threadList.removeThread(this);
            try {
                clientSocket.close();
            } catch (IOException e) {
                System.out.println("Error closing socket: " + e.getMessage());
            }
        }
    }
}

```

ThreadList.java Source Code:

```
import java.util.ArrayList;
import java.util.List;

public class ThreadList {
    private List<EchoServerThread> threadlist;

    public ThreadList() {
        threadlist = new ArrayList<EchoServerThread>();
    }

    public synchronized int getNumberOfThreads() {
        return threadlist.size();
    }

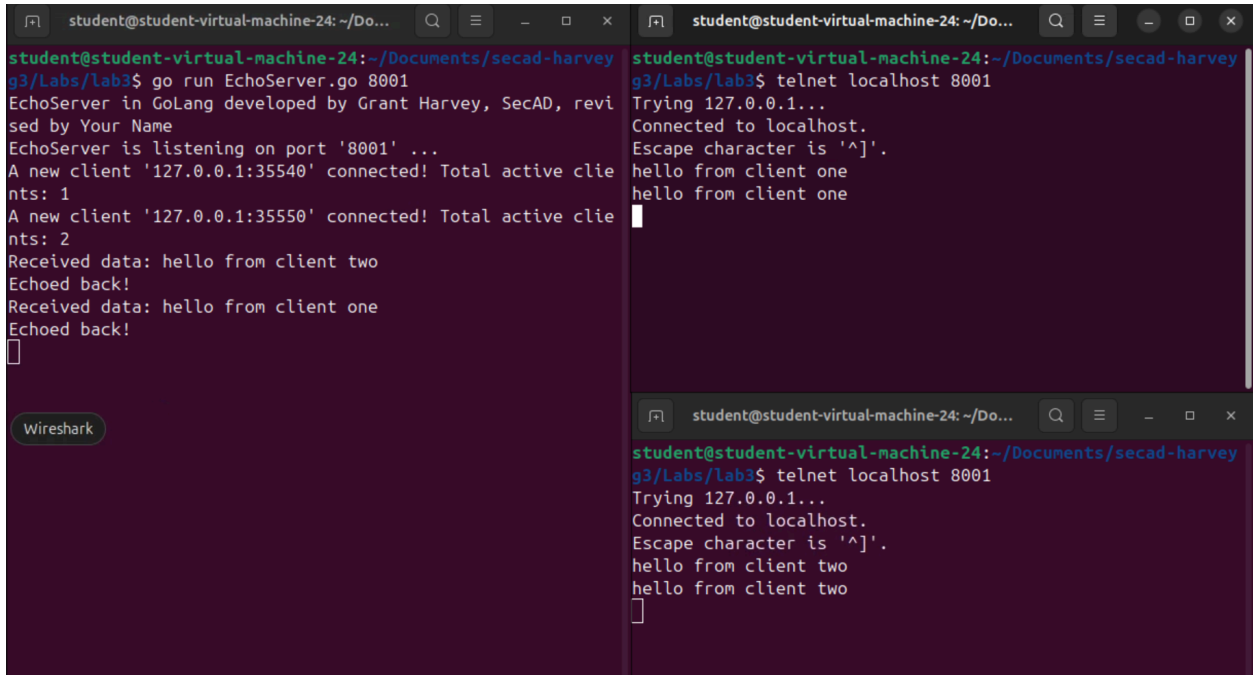
    public synchronized void addThread(EchoServerThread newthread) {
        threadlist.add(newthread);
        System.out.println("Active client threads: " + getNumberOfThreads());
    }

    public synchronized void removeThread(EchoServerThread thread) {
        threadlist.remove(thread);
        System.out.println("Client disconnected. Active client threads: " + getNumberOfThreads());
    }

    public synchronized void sendToAll(String message) {
        for (EchoServerThread thread : threadlist) {
            thread.sendMessage(message);
        }
    }
}
```

Task 2 - Concurrent Server Programming in GoLang

a.



```
student@student-virtual-machine-24: ~/Documents/secad-harvey
g3/Labs/lab3$ go run EchoServer.go 8001
EchoServer in GoLang developed by Grant Harvey, SecAD, revised by Your Name
EchoServer is listening on port '8001' ...
A new client '127.0.0.1:35540' connected! Total active clients: 1
A new client '127.0.0.1:35550' connected! Total active clients: 2
Received data: hello from client two
Echoed back!
Received data: hello from client one
Echoed back!

```

```
student@student-virtual-machine-24: ~/Documents/secad-harvey
g3/Labs/lab3$ telnet localhost 8001
Trying 127.0.0.1...
Connected to localhost.
Escape character is '^]'.
hello from client one
hello from client one

```

```
student@student-virtual-machine-24: ~/Documents/secad-harvey
g3/Labs/lab3$ telnet localhost 8001
Trying 127.0.0.1...
Connected to localhost.
Escape character is '^]'.
hello from client two
hello from client two

```

This EchoServer in Go accepts incoming TCP connections in separate goroutines and safely updates and prints the active client count using atomic package operations whenever a client connects or disconnects.

EchoServer.go Source Code:

```
/* Simple EchoServer in GoLang by Phu Phung, customized by Grant Harvey for SecAD*/
package main
```

```
import (
    "fmt"
    "net"
    "os"
    "sync/atomic"
)

const BUFFERSIZE int = 1024

// Global counter for connected clients
var client_count int64

func main() {
    if len(os.Args) != 2 {
        fmt.Printf("Usage: %s <port>\n", os.Args[0])
        os.Exit(0)
    }
    port := os.Args[1]
    if len(port) > 5 {
        fmt.Println("Invalid port value. Try again!")
        os.Exit(1)
    }
    server, err := net.Listen("tcp", ":"+port)
    if err != nil {
        fmt.Printf("Cannot listen on port '"+port+"'!\n", port)
    }
}
```

```

    os.Exit(2)
}
fmt.Println("EchoServer in GoLang developed by Grant Harvey, SecAD, revised by Your Name")
fmt.Printf("EchoServer is listening on port '%s' ...\n", port)

for {
    client_conn, err := server.Accept()
    if err != nil {
        continue
    }
    go client_goroutine(client_conn)
}
}

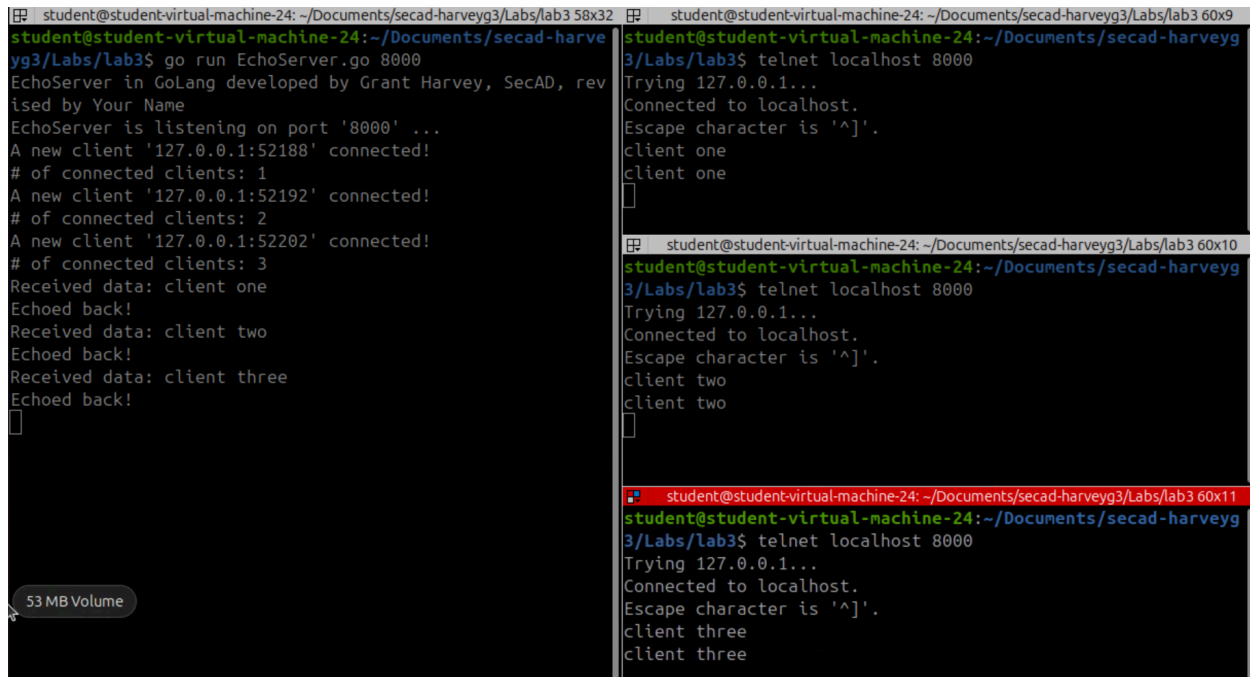
func client_goroutine(client_conn net.Conn) {
    // Increment client count when connection opens
    atomic.AddInt64(&client_count, 1)
    currentCount := atomic.LoadInt64(&client_count)
    fmt.Printf("A new client '%s' connected! Total active clients: %d\n", client_conn.RemoteAddr().String(), currentCount)

    // Ensure connection closes and counter decrements when client disconnects
    defer func() {
        client_conn.Close()
        atomic.AddInt64(&client_count, -1)
        remaining := atomic.LoadInt64(&client_count)
        fmt.Printf("Client '%s' disconnected. Total active clients: %d\n", client_conn.RemoteAddr().String(), remaining)
    }()

    var buffer [BUFFERSIZE]byte
    for {
        byte_received, read_err := client_conn.Read(buffer[0:])
        if read_err != nil {
            fmt.Println("Error in receiving (client disconnected)...")
            return
        }
        _, write_err := client_conn.Write(buffer[0:byte_received])
        if write_err != nil {
            fmt.Println("Error in sending...")
            return
        }
        fmt.Printf("Received data: %sEchoed back!\n", string(buffer[0:byte_received]))
    }
}

```

b.



The screenshot shows a terminal window with two panes. The left pane shows the output of a Go program named 'EchoServer'. The right pane shows the output of a telnet client connecting to the server.

```
student@student-virtual-machine-24: ~/Documents/secad-harveyg3/Labs/lab3 58x32
student@student-virtual-machine-24:~/Documents/secad-harveyg3/Labs/lab3$ go run EchoServer.go 8000
EchoServer in GoLang developed by Grant Harvey, SecAD, revised by Your Name
EchoServer is listening on port '8000' ...
A new client '127.0.0.1:52188' connected!
# of connected clients: 1
A new client '127.0.0.1:52192' connected!
# of connected clients: 2
A new client '127.0.0.1:52202' connected!
# of connected clients: 3
Received data: client one
Echoed back!
Received data: client two
Echoed back!
Received data: client three
Echoed back!

```

```
student@student-virtual-machine-24: ~/Documents/secad-harveyg3/Labs/lab3 60x9
student@student-virtual-machine-24:~/Documents/secad-harveyg3/Labs/lab3$ telnet localhost 8000
Trying 127.0.0.1...
Connected to localhost.
Escape character is '^]'.
client one
client one

```

```
student@student-virtual-machine-24: ~/Documents/secad-harveyg3/Labs/lab3 60x10
student@student-virtual-machine-24:~/Documents/secad-harveyg3/Labs/lab3$ telnet localhost 8000
Trying 127.0.0.1...
Connected to localhost.
Escape character is '^]'.
client two
client two

```

```
student@student-virtual-machine-24: ~/Documents/secad-harveyg3/Labs/lab3 60x11
student@student-virtual-machine-24:~/Documents/secad-harveyg3/Labs/lab3$ telnet localhost 8000
Trying 127.0.0.1...
Connected to localhost.
Escape character is '^]'.
client three
client three

```

Source Code Snippets:

```
...
var newclient = make(chan net.Conn)
...
go func() {
    client_count := 0
    for {
        select {
            case conn := <-newclient:
                client_count++
                fmt.Printf("A new client '%s' connected!\n", conn.RemoteAddr().String())
                fmt.Printf("# of connected clients: %d\n", client_count)
        }
    }
}()

for {
    client_conn, err := server.Accept()
    if err != nil {
        continue
    }
    go func(c net.Conn) {
        newclient <- c
    }(client_conn)

    go client_goroutine(client_conn)
}

func client_goroutine(client_conn net.Conn) {
    defer client_conn.Close()

    var buffer [BUFFERSIZE]byte
```

```

for {
    byte_received, read_err := client_conn.Read(buffer[0:])
    if read_err != nil {
        fmt.Println("Error in receiving (client disconnected)...")
        return
    }
    write_err := client_conn.Write(buffer[0:byte_received])
    if write_err != nil {
        fmt.Println("Error in sending...")
        return
    }
    fmt.Printf("Received data: %sEchoed back!\n", string(buffer[0:byte_received]))
}
}
}

```

C.

```

student@student-virtual-machine-24: ~/Documents/secad-harveyg3/Labs/lab3 58x32
student@student-virtual-machine-24:~/Documents/secad-harveyg3/Labs/lab3$ go run BroadcastEchoServer.go 8000
EchoServer in GoLang developed by Phu Phung, SecAD, revised by Grant Harvey
EchoServer is listening on port '8000' ...
A new client '127.0.0.1:52530' connected!
# of connected clients: 1
Send data: A new client '127.0.0.1:52530' connected!
# of connected clients: 1
to all clients!
A new client '127.0.0.1:43474' connected!
# of connected clients: 2
Send data: A new client '127.0.0.1:43474' connected!
# of connected clients: 2
to all clients!
A new client '127.0.0.1:43480' connected!
# of connected clients: 3
Send data: A new client '127.0.0.1:43480' connected!
# of connected clients: 3
to all clients!
Send data: client one
to all clients!
Send data: client two
to all clients!
Send data: client three
to all clients!

```

```

student@student-virtual-machine-24: ~/Documents/secad-harveyg3/Labs/lab3 60x9
A new client '127.0.0.1:43474' connected!
# of connected clients: 2
A new client '127.0.0.1:43480' connected!
# of connected clients: 3
client one
client one
client two
client three

```

```

student@student-virtual-machine-24: ~/Documents/secad-harveyg3/Labs/lab3 60x10
Escape character is '^]'.
A new client '127.0.0.1:43474' connected!
# of connected clients: 2
A new client '127.0.0.1:43480' connected!
# of connected clients: 3
client one
client two
client two
client three

```

```

student@student-virtual-machine-24: ~/Documents/secad-harveyg3/Labs/lab3 60x11
3/Labs/lab3$ telnet localhost 8000
Trying 127.0.0.1...
Connected to localhost.
Escape character is '^]'.
A new client '127.0.0.1:43480' connected!
# of connected clients: 3
client one
client two
client three
client three

```

The BroadcastEchoServer.go program uses a synchronized map of active TCP sockets and a sendtoAll function to broadcast system connection notices and incoming client messages to every connected terminal simultaneously.

Source Code:

```

/* Simple EchoServer in GoLang by Phu Phung, customized by <YOUR NAME> for SecAD */
package main

import (
    "fmt"
    "net"
    "os"
    "sync"
)

const BUFFERSIZE int = 1024

```

```

var newclient = make(chan net.Conn)
var allClient_conns = make(map[net.Conn]string)
var mapMutex sync.RWMutex

func main() {
    if len(os.Args) != 2 {
        fmt.Printf("Usage: %s <port>\n", os.Args[0])
        os.Exit(0)
    }
    port := os.Args[1]
    if len(port) > 5 {
        fmt.Println("Invalid port value. Try again!")
        os.Exit(1)
    }
    server, err := net.Listen("tcp", ":"+port)
    if err != nil {
        fmt.Printf("Cannot listen on port '%s'\n", port)
        os.Exit(2)
    }
    fmt.Println("EchoServer in GoLang developed by Phu Phung, SecAD, revised by Grant Harvey")
    fmt.Printf("EchoServer is listening on port '%s' ...\n", port)

    go monitorClients()

    for {
        client_conn, err := server.Accept()
        if err != nil {
            continue
        }
        newclient <- client_conn
    }
}

func monitorClients() {
    for {
        select {
        case client_conn := <-newclient:
            mapMutex.Lock()
            allClient_conns[client_conn] = client_conn.RemoteAddr().String()
            mapMutex.Unlock()

            welcomemessage := fmt.Sprintf("A new client '%s' connected!\n# of connected clients: %d\n",
client_conn.RemoteAddr().String(), len(allClient_conns))
            fmt.Print(welcomemessage)
            sendtoAll([]byte(welcomemessage))
            go client_goroutine(client_conn)
        }
    }
}

func client_goroutine(client_conn net.Conn) {
    defer func() {
        client_conn.Close()
        mapMutex.Lock()
        delete(allClient_conns, client_conn)
        mapMutex.Unlock()
    }()

    var buffer [BUFFERSIZE]byte
    for {

```

```

    byte_received, read_err := client_conn.Read(buffer[0:])
    if read_err != nil {
        fmt.Println("Error in receiving (client disconnected)...")
        return
    }

    // Broadcast received message to all connected clients
    sendtoAll(buffer[0:byte_received])
}

func sendtoAll(data []byte) {
    mapMutex.RLock()
    defer mapMutex.RUnlock()

    for client_conn, _ := range allClient_conns {
        _, write_err := client_conn.Write(data)
        if write_err != nil {
            fmt.Println("Error in sending...")
            continue
        }
    }
    fmt.Printf("Send data: %s to all clients!\n", string(data))
}

```

d.

The image shows two terminal windows side-by-side. The left window shows the output of a Go program running 'BroadcastEchoServer.go 8000'. The right window shows telnet connections to localhost 8000.

```

student@student-virtual-machine-24: ~/Documents/secad-harveyg3/Labs/lab3 58x32
student@student-virtual-machine-24:~/Documents/secad-harveyg3/Labs/lab3$ go run BroadcastEchoServer.go 8000
EchoServer in GoLang developed by Phu Phung, SecAD, revised by <YOUR NAME>
EchoServer is listening on port '8000' ...
A new client '127.0.0.1:58152' connected!
# of connected clients: 1
Send data: A new client '127.0.0.1:58152' connected!
# of connected clients: 1
to all clients!
A new client '127.0.0.1:58160' connected!
# of connected clients: 2
Send data: A new client '127.0.0.1:58160' connected!
# of connected clients: 2
to all clients!
A new client '127.0.0.1:58172' connected!
# of connected clients: 3
Send data: A new client '127.0.0.1:58172' connected!
# of connected clients: 3
to all clients!
Send data: Client One
to all clients!
Error in receiving...
Client '127.0.0.1:58152' is DISCONNECTED!
# of connected clients: 2
Send data: Client '127.0.0.1:58152' is DISCONNECTED!
# of connected clients: 2
to all clients!

```

```

student@student-virtual-machine-24: ~/Documents/secad-harveyg3/Labs/lab3 60x20
student@student-virtual-machine-24:~/Documents/secad-harveyg3/Labs/lab3$ telnet localhost 8000
Trying 127.0.0.1...
Connected to localhost.
Escape character is '^]'.
A new client '127.0.0.1:58160' connected!
# of connected clients: 2
A new client '127.0.0.1:58172' connected!
# of connected clients: 3
Client One
Client One
Client '127.0.0.1:58152' is DISCONNECTED!
# of connected clients: 2

```

```

student@student-virtual-machine-24: ~/Documents/secad-harveyg3/Labs/lab3 60x11
student@student-virtual-machine-24:~/Documents/secad-harveyg3/Labs/lab3$ telnet localhost 8000
Trying 127.0.0.1...
Connected to localhost.
Escape character is '^]'.
A new client '127.0.0.1:58172' connected!
# of connected clients: 3
Client One
Client '127.0.0.1:58152' is DISCONNECTED!
# of connected clients: 2

```

It avoids data races because all adding and removing of clients is routed through channels to one central loop, while mapMutex stops goroutines from reading and writing to the connection list at the same time.

Task 3 - Asynchronous Programming in Node.js

a.

- i. Using `require('./module1')` imports the variables and functions exported by `module1.js` so `index.js` can access them as properties of the `m1` object.

```
student@student-virtual-machine-24:~/Documents/secad-harveyg3/Labs/lab3$ cat module1.js
var a = 1;
module.exports.a = a;
exports.f = function() { console.log('f from module1'); };
student@student-virtual-machine-24:~/Documents/secad-harveyg3/Labs/lab3$ cat index.js
console.log("Lab 3-3.a.i by Grant Harvey. Test the module inclusion...");
var m1 = require('./module1');
console.log('m1=' + m1);
console.log('m1.a=' + m1.a);
m1.f();
student@student-virtual-machine-24:~/Documents/secad-harveyg3/Labs/lab3$ node index.js
Lab 3-3.a.i by Grant Harvey. Test the module inclusion...
m1=[object Object]
m1.a=1
f from module1
student@student-virtual-machine-24:~/Documents/secad-harveyg3/Labs/lab3$
```

- ii. The `watcher.js` loops asynchronously executing the `fs.watch` function whenever the operating system detects a file system modification on `target.txt`

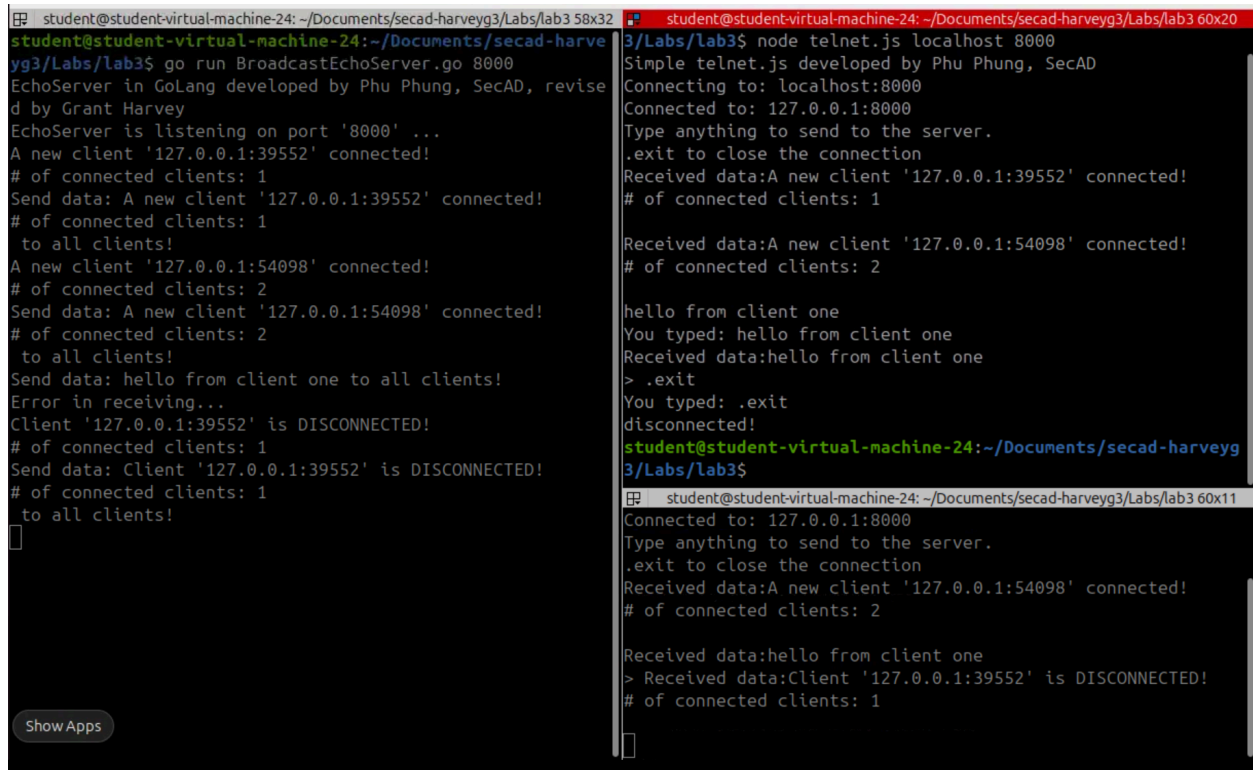
```
student@student-virtual-machine-24:~/Documents/secad-harveyg3/Labs/lab3$ node watcher.js
Lab 3-3.a.ii by Grant Harvey. Watching target.txt for changes...
File 'target.txt' just changed!
File 'target.txt' just changed!
```

b.

i.

<pre>student@student-virtual-machine-24:~/Documents/secad-harveyg3/Labs/lab3 58x32 student@student-virtual-machine-24:~/Documents/secad-harveyg3/Labs/lab3\$ go run BroadcastEchoServer.go 8000 EchoServer in GoLang developed by Phu Phung, SecAD, revised by Grant Harvey EchoServer is listening on port '8000' ... A new client '127.0.0.1:39604' connected! # of connected clients: 1 Send data: A new client '127.0.0.1:39604' connected! # of connected clients: 1 to all clients!</pre>	<pre>student@student-virtual-machine-24:~/Documents/secad-harveyg3/Labs/lab3 60x20 student@student-virtual-machine-24:~/Documents/secad-harveyg3/Labs/lab3\$ node telnet.js localhost 8000 Simple telnet.js developed by Phu Phung, SecAD Connecting to: localhost:8000 Connected to: 127.0.0.1:8000 Type anything to send to the server. .exit to close the connection Received data:A new client '127.0.0.1:39604' connected! # of connected clients: 1</pre>
---	---

- ii. The program listens for keyboard input line-by-line using `readline`, sending the string over the TCP socket with `client.write()` or destroying the connection when `.exit` is typed.



The image shows two terminal windows side-by-side. The left window is a Go application running a broadcast echo server. The right window is a Node.js application running a telnet client that connects to the server on localhost:8000.

```
student@student-virtual-machine-24: ~/Documents/secad-harveyg3/Labs/lab3 58x32
student@student-virtual-machine-24:~/Documents/secad-harveyg3/Labs/lab3$ go run BroadcastEchoServer.go 8000
EchoServer in Golang developed by Phu Phung, SecAD, revised by Grant Harvey
EchoServer is listening on port '8000' ...
A new client '127.0.0.1:39552' connected!
# of connected clients: 1
Send data: A new client '127.0.0.1:39552' connected!
# of connected clients: 1
to all clients!
A new client '127.0.0.1:54098' connected!
# of connected clients: 2
Send data: A new client '127.0.0.1:54098' connected!
# of connected clients: 2
to all clients!
Send data: hello from client one to all clients!
Error in receiving...
Client '127.0.0.1:39552' is DISCONNECTED!
# of connected clients: 1
Send data: Client '127.0.0.1:39552' is DISCONNECTED!
# of connected clients: 1
to all clients!
[ ]
```

Show Apps

```
student@student-virtual-machine-24: ~/Documents/secad-harveyg3/Labs/lab3 60x20
3/Labs/lab3$ node telnet.js localhost 8000
Simple telnet.js developed by Phu Phung, SecAD
Connecting to: localhost:8000
Connected to: 127.0.0.1:8000
Type anything to send to the server.
.exit to close the connection
Received data:A new client '127.0.0.1:39552' connected!
# of connected clients: 1

Received data:A new client '127.0.0.1:54098' connected!
# of connected clients: 2

hello from client one
You typed: hello from client one
Received data:hello from client one
> .exit
You typed: .exit
disconnected!
student@student-virtual-machine-24:~/Documents/secad-harveyg3/Labs/lab3$

student@student-virtual-machine-24: ~/Documents/secad-harveyg3/Labs/lab3 60x11
Connected to: 127.0.0.1:8000
Type anything to send to the server.
.exit to close the connection
Received data:A new client '127.0.0.1:54098' connected!
# of connected clients: 2

Received data:hello from client one
> Received data:Client '127.0.0.1:39552' is DISCONNECTED!
# of connected clients: 1
[ ]
```